

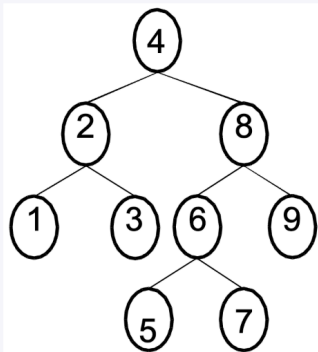
Resumo do Código Implementado

Este notebook implementa os três percursos em profundidade pedidos no enunciado para a árvore binária apresentada na imagem

`ArvoresBinarias_AutoBalanceadas.png` :

Implemente um algoritmo que visite todos os nós de uma árvore binária com uma busca em profundidade em-ordem, pré-ordem e pós-ordem.

- Exemplo, para a árvore abaixo a saída deve ser:
 - Pré-ordem: 4, 2, 1, 3, 8, 6, 5, 7, 9
 - Em-ordem: 1, 2, 3, 4, 5, 6, 7, 8, 9
 - Pós-ordem: 1, 3, 2, 5, 7, 6, 9, 8, 4



pré-ordem, em-ordem e pós-ordem. A árvore é construída manualmente com nós ligados por referências para filho esquerdo e filho direito, mantendo exatamente a estrutura do exemplo do exercício.

Cada percurso foi implementado em uma célula própria, com uma função recursiva independente. Depois, há células separadas para visualizar didaticamente a execução de cada tipo de busca, usando uma representação SVG da árvore com destaque do nó visitado em cada passo.

Estrutura simplificada do notebook

- Importações e definição da estrutura de nó.
- Construção da árvore binária do enunciado.
- Implementação da busca em profundidade em-ordem.
- Implementação da busca em profundidade pré-ordem.
- Implementação da busca em profundidade pós-ordem.
- Execução e validação dos resultados.
- Funções auxiliares para visualização didática.
- Visualização da pré-ordem, em-ordem e pós-ordem em células separadas.
- Resultados finais pedidos no enunciado.

```
In [1]: # Descrição: esta célula importa apenas os recursos necessários para
```

```
# e exibir a visualização didática em HTML/SVG dentro do notebook.

from dataclasses import dataclass
from IPython.display import HTML, display
```

In [2]: *# Descrição: esta célula define a estrutura básica de um nó da árvore*
Cada nó armazena um valor e, opcionalmente, referências para o filho
e para o filho direito.

```
@dataclass
class No:
    valor: int
    esquerda: "No | None" = None
    direita: "No | None" = None
```

In [3]: *# Descrição: esta célula constrói exatamente a árvore do enunciado.*
Estrutura:

```
#           4
#         /  \
#        2    8
#       / \  / \
#      1  3 6  9
#         / \
#        5  7
```

```
raiz = No(4)
raiz.esquerda = No(2, No(1), No(3))
raiz.direita = No(8, No(6, No(5), No(7)), No(9))

raiz
```

Out[3]: No(valor=4, esquerda=No(valor=2, esquerda=No(valor=1, esquerda=None, direita=None), direita=No(valor=3, esquerda=None, direita=None)), direita=No(valor=8, esquerda=No(valor=6, esquerda=No(valor=5, esquerda=None, direita=None), direita=No(valor=7, esquerda=None, direita=None)), direita=No(valor=9, esquerda=None, direita=None)))

Busca em profundidade em-ordem

In [4]: *# Descrição: esta célula implementa a busca em profundidade em-ordem.*
Regra: visitar a subárvore esquerda, depois o nó atual, depois a
Complexidade: O(n) no tempo, pois cada nó é visitado exatamente u

```
def busca_em_ordem(no):
    if no is None:
        return []

    return (
        busca_em_ordem(no.esquerda)
        + [no.valor]
        + busca_em_ordem(no.direita)
    )
```

Busca em profundidade pré-ordem

```
In [5]: # Descrição: esta célula implementa a busca em profundidade pré-ordem
# Regra: visitar o nó atual, depois a subárvore esquerda, depois a
# Complexidade:  $O(n)$  no tempo, pois cada nó é visitado exatamente u

def busca_pre_ordem(no):
    if no is None:
        return []

    return (
        [no.valor]
        + busca_pre_ordem(no.esquerda)
        + busca_pre_ordem(no.direita)
    )
```

Busca em profundidade pós-ordem

```
In [6]: # Descrição: esta célula implementa a busca em profundidade pós-ordem
# Regra: visitar a subárvore esquerda, depois a subárvore direita,
# Complexidade:  $O(n)$  no tempo, pois cada nó é visitado exatamente u

def busca_pos_ordem(no):
    if no is None:
        return []

    return (
        busca_pos_ordem(no.esquerda)
        + busca_pos_ordem(no.direita)
        + [no.valor]
    )
```

```
In [7]: # Descrição: esta célula executa as três buscas e valida se as saídas
# com os resultados apresentados no enunciado do exercício.

resultado_pre_ordem = busca_pre_ordem(raiz)
resultado_em_ordem = busca_em_ordem(raiz)
resultado_pos_ordem = busca_pos_ordem(raiz)

assert resultado_pre_ordem == [4, 2, 1, 3, 8, 6, 5, 7, 9]
assert resultado_em_ordem == [1, 2, 3, 4, 5, 6, 7, 8, 9]
assert resultado_pos_ordem == [1, 3, 2, 5, 7, 6, 9, 8, 4]

print("Pré-ordem:", resultado_pre_ordem)
print("Em-ordem:", resultado_em_ordem)
print("Pós-ordem:", resultado_pos_ordem)
```

Pré-ordem: [4, 2, 1, 3, 8, 6, 5, 7, 9]

Em-ordem: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Pós-ordem: [1, 3, 2, 5, 7, 6, 9, 8, 4]

Visualização didática dos percursos

In [8]: *# Descrição: esta célula cria funções auxiliares para gerar os passos dos percursos. A implementação abaixo é usada apenas para explicação ficando separada das células que implementam as buscas em si.*

```
POSICOES = {
    4: (220, 35),
    2: (120, 115),
    8: (320, 115),
    1: (55, 205),
    3: (185, 205),
    6: (275, 205),
    9: (385, 205),
    5: (235, 295),
    7: (315, 295),
}

ARESTAS = [(4, 2), (4, 8), (2, 1), (2, 3), (8, 6), (8, 9), (6, 5),

def gerar_passos_visualizacao(no, modo):
    passos = []
    visitados = []

    def visitar(atual):
        if atual is None:
            return

        if modo == "pre":
            visitados.append(atual.valor)
            passos.append((atual.valor, visitados.copy()))

        visitar(atual.esquerda)

        if modo == "em":
            visitados.append(atual.valor)
            passos.append((atual.valor, visitados.copy()))

        visitar(atual.direita)

        if modo == "pos":
            visitados.append(atual.valor)
            passos.append((atual.valor, visitados.copy()))

    visitar(no)
    return passos
```

In [9]: *# Descrição: esta célula desenha a árvore em SVG para cada passo do algoritmo. O nó atual fica destacado em laranja; os nós já visitados ficam em cinza.*

```
def desenhar_arvore_svg(no_atual=None, visitados=None):
    visitados = set(visitados or [])
    partes = [
        '<svg width="440" height="350" viewBox="0 0 440 350" '
        'style="background:#ffffff;border:1px solid #d0d7de;border-
```

```

for origem, destino in ARESTAS:
    x1, y1 = POSICOES[origem]
    x2, y2 = POSICOES[destino]
    partes.append(
        f'<line x1="{x1}" y1="{y1}" x2="{x2}" y2="{y2}" '
        f'stroke="#24292f" stroke-width="3" />'
    )

for valor, (x, y) in POSICOES.items():
    if valor == no_atual:
        cor = "#f59e0b"
        largura = 5
    elif valor in visitados:
        cor = "#22c55e"
        largura = 4
    else:
        cor = "#f8fafc"
        largura = 3

    partes.append(
        f'<circle cx="{x}" cy="{y}" r="30" fill="{cor}" '
        f'stroke="#111827" stroke-width="{largura}" />'
    )
    partes.append(
        f'<text x="{x}" y="{y + 11}" text-anchor="middle" '
        f'font-size="34" font-family="Arial, sans-serif" fill="#"
        f'{valor}</text>'
    )

partes.append("</svg>")
return "".join(partes)

def mostrar_execucao_visual(titulo, passos):
    blocos = []
    for indice, (no_atual, visitados) in enumerate(passos, start=1):
        sequencia = ", ".join(map(str, visitados))
        blocos.append(
            f'<div style="display:inline-block;margin:0 14px 18px 0;'
            f'<div style="font-family:Arial,sans-serif;font-weight:'
            + desenhara_arvore_svg(no_atual, visitados)
            + f'<div style="font-family:Arial,sans-serif;margin-top'
            + '</div>'
        )

    legenda = (
        f'<div style="font-family:Arial,sans-serif;margin:8px 0 16px'
        f'<span style="display:inline-block;width:14px;height:14px;b'
        f'<span style="display:inline-block;width:14px;height:14px;b'
        f'<span style="display:inline-block;width:14px;height:14px;b'
        f'</div>'
    )

    html = f'<h3 style="font-family:Arial,sans-serif">{titulo}</h3>'
    display(HTML(html))

```

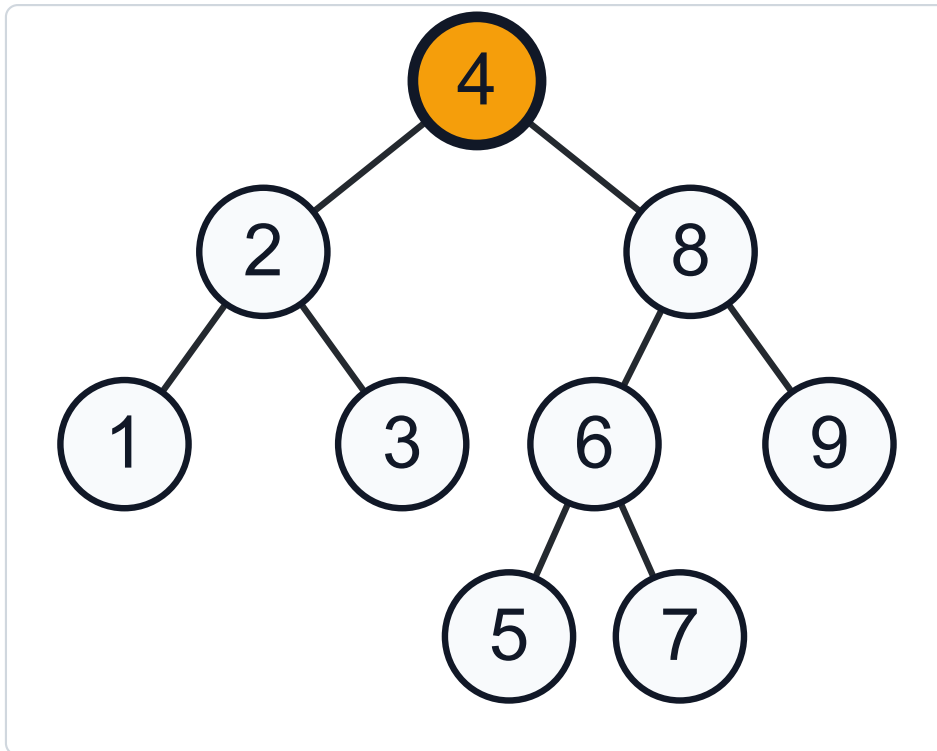
```
In [10]: # Descrição: esta célula mostra visualmente a execução da busca pré
# A regra visualizada é: nó atual, esquerda, direita.

passos_pre_ordem = gerar_passos_visualizacao(raiz, "pre")
mostrar_execucao_visual("Execução visual da busca pré-ordem", passo
```

Execução visual da busca pré-ordem

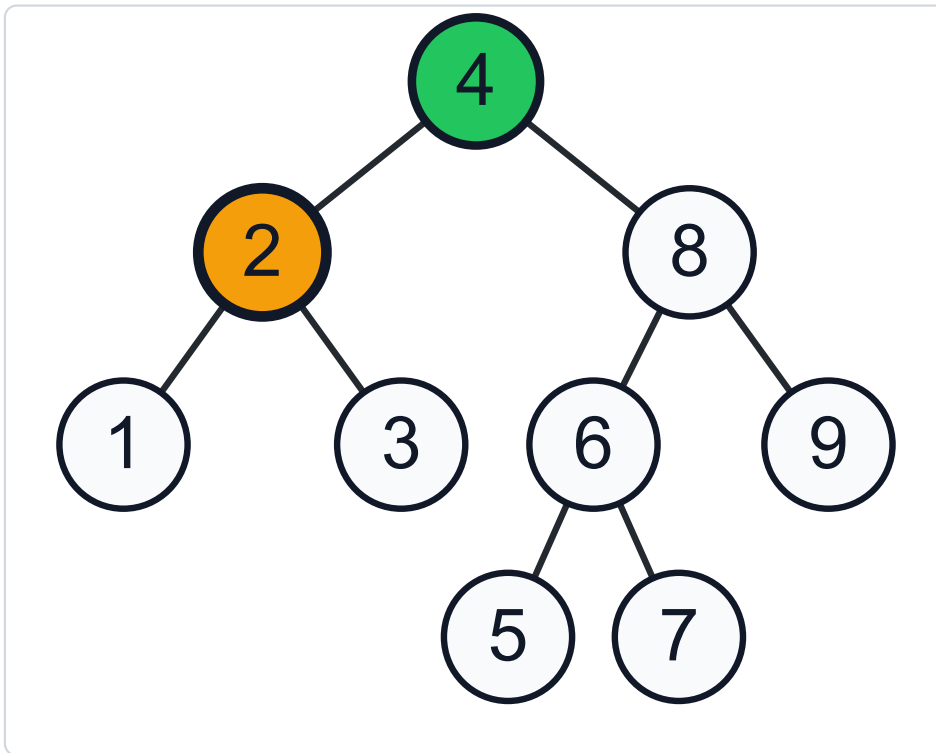
■ nó atual ■ já visitado □ ainda não visitado

Passo 1: visita 4



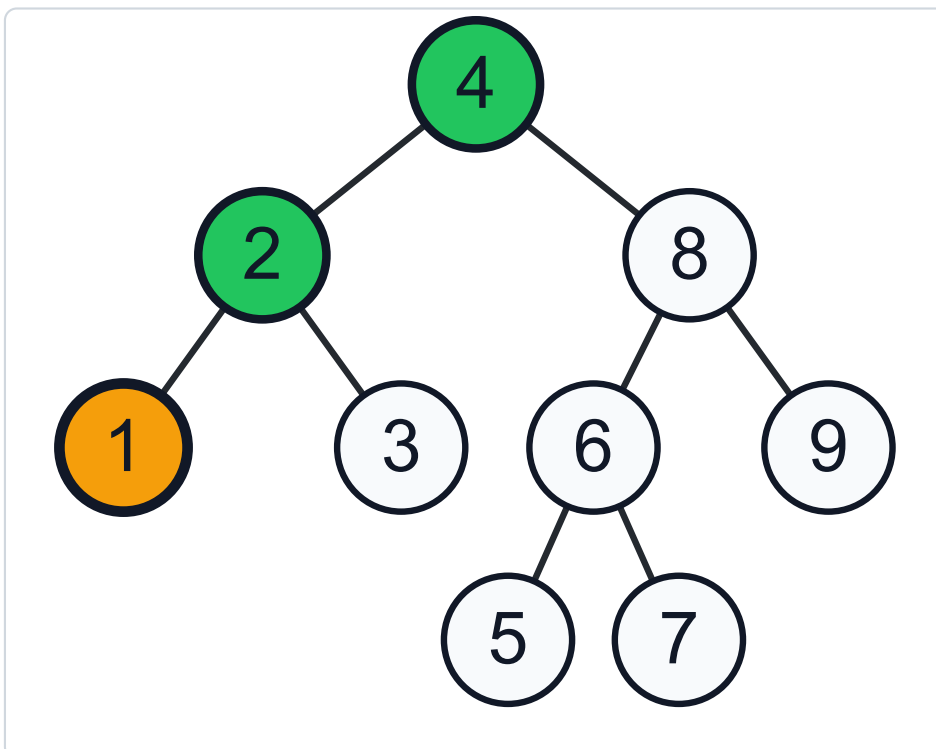
Sequência: 4

Passo 2: visita 2



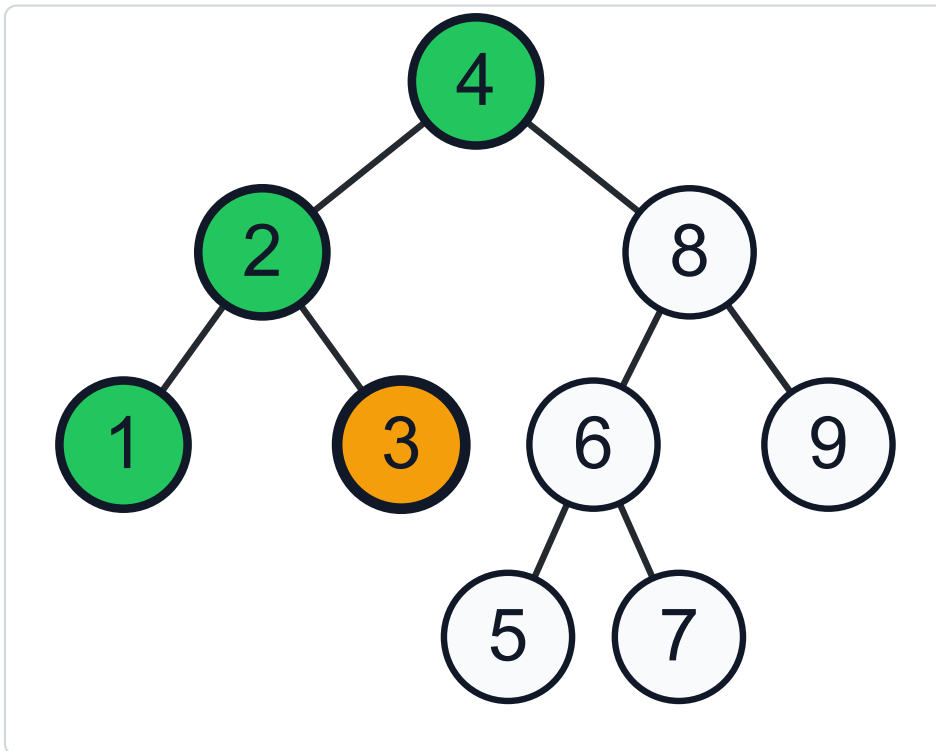
Sequência: 4, 2

Passo 3: visita 1



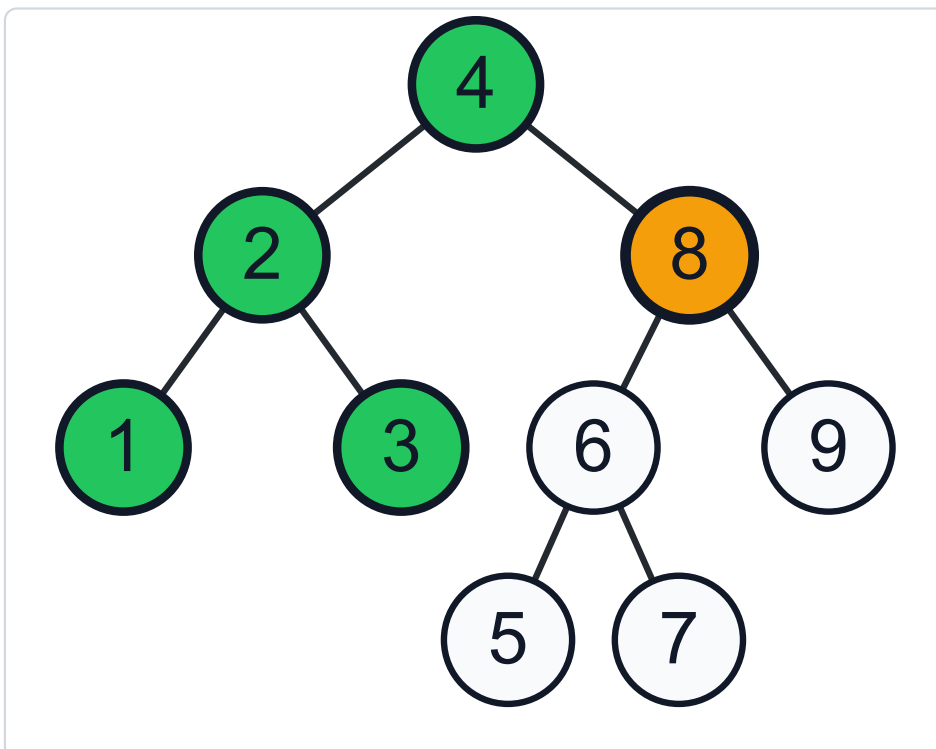
Sequência: 4, 2, 1

Passo 4: visita 3



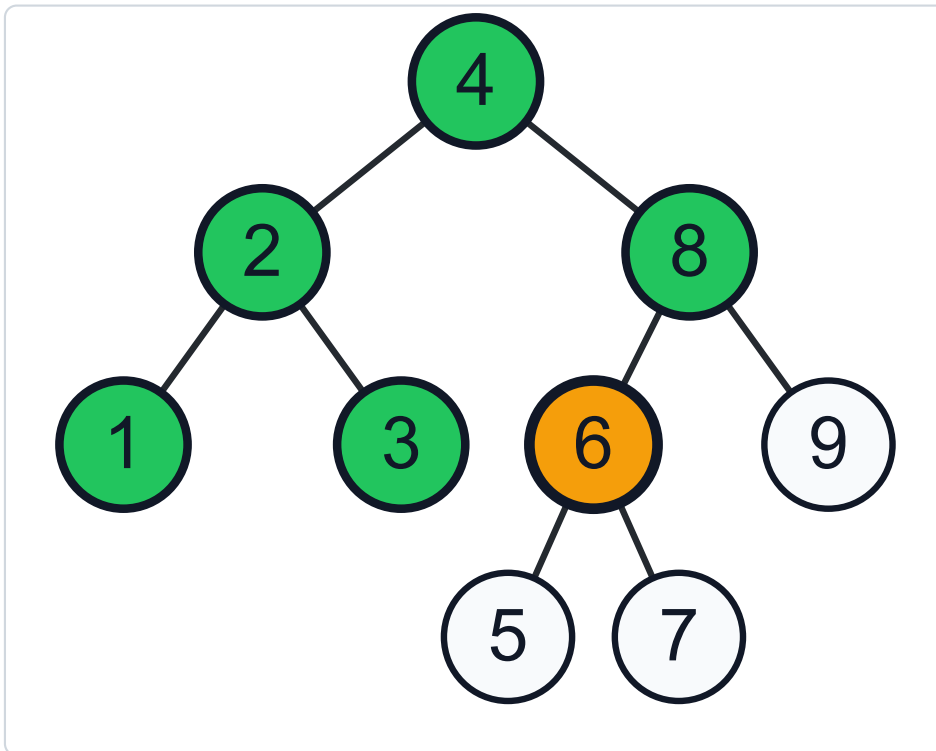
Sequência: 4, 2, 1, 3

Passo 5: visita 8



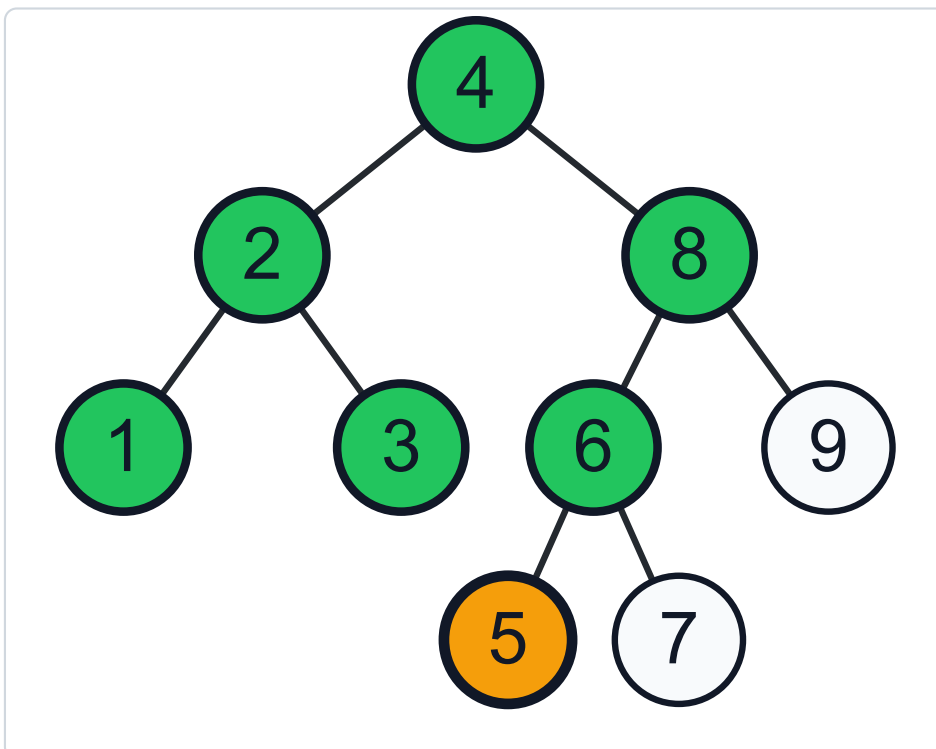
Sequência: 4, 2, 1, 3, 8

Passo 6: visita 6



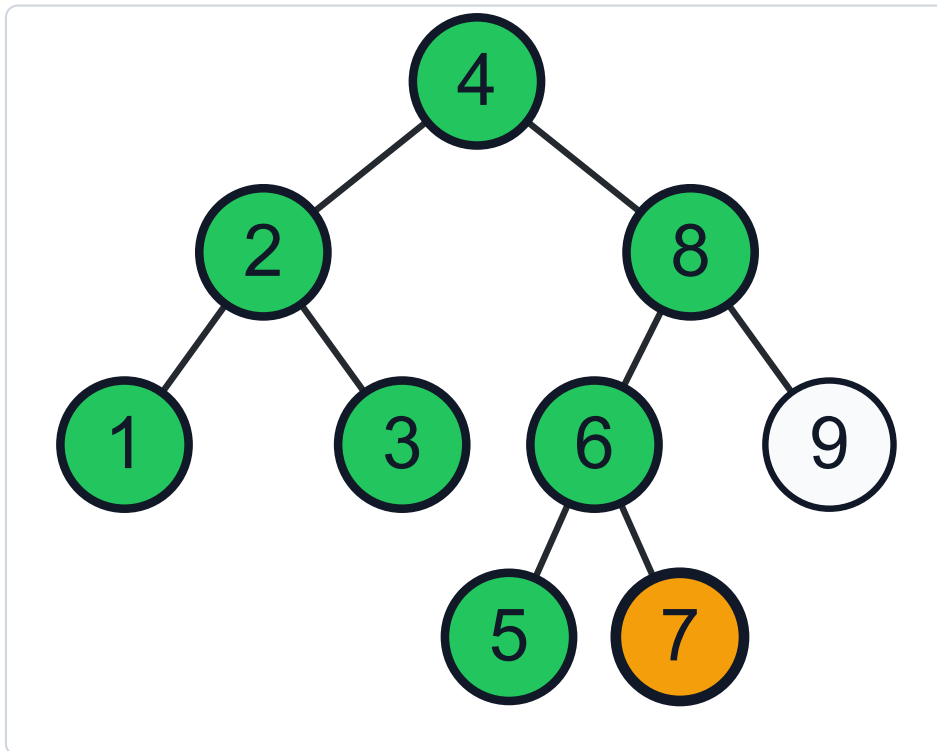
Sequência: 4, 2, 1, 3, 8, 6

Passo 7: visita 5



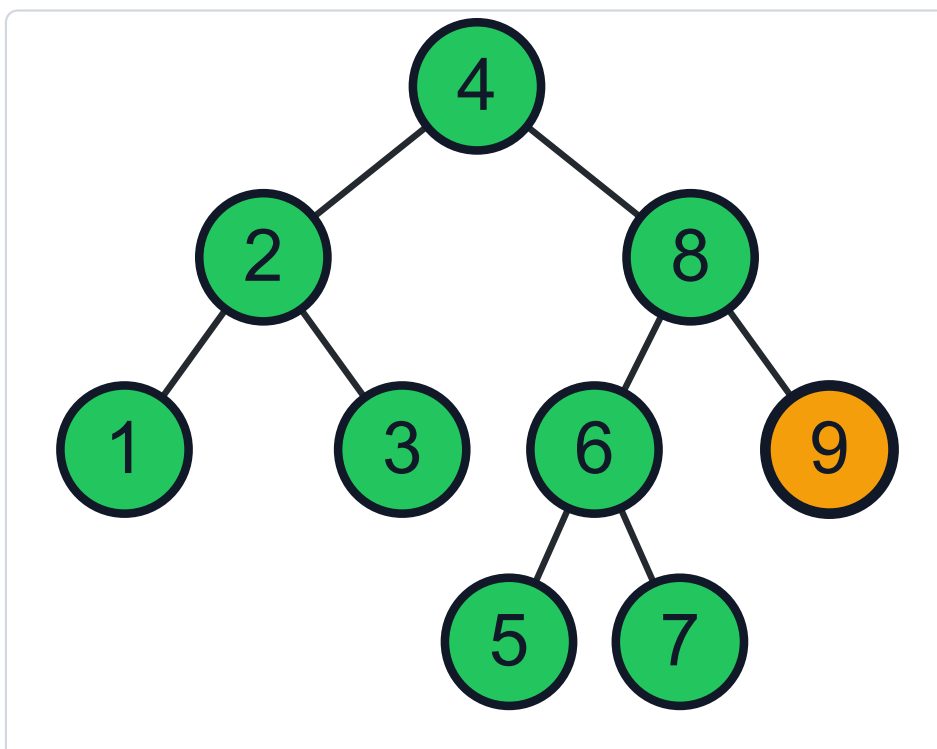
Sequência: 4, 2, 1, 3, 8, 6, 5

Passo 8: visita 7



Sequência: 4, 2, 1, 3, 8, 6, 5, 7

Passo 9: visita 9



Sequência: 4, 2, 1, 3, 8, 6, 5, 7, 9

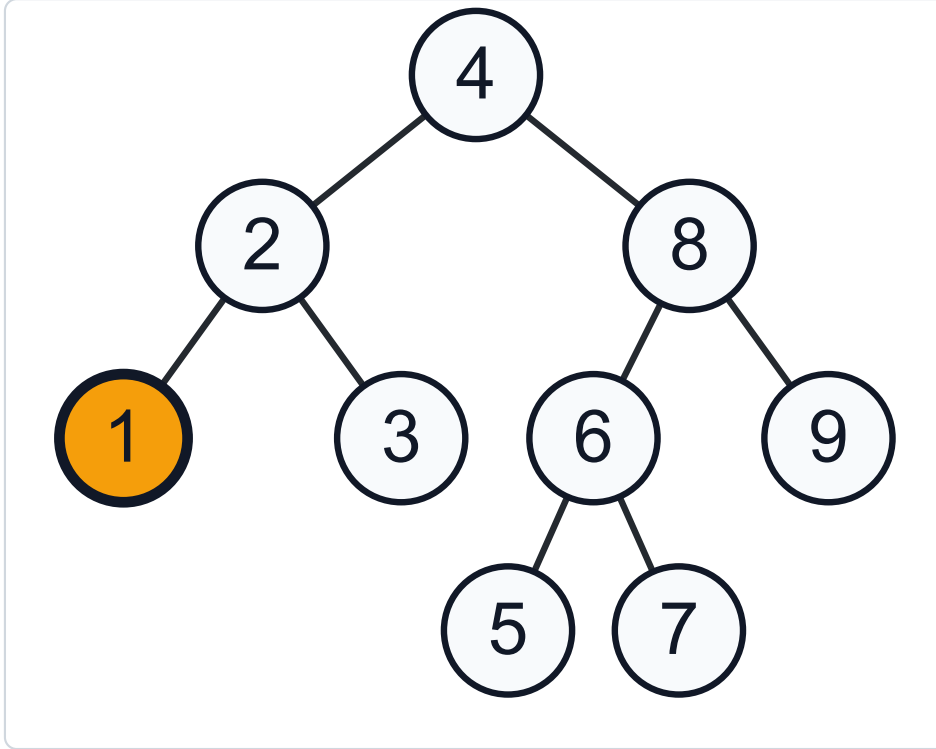
```
In [11]: # Descrição: esta célula mostra visualmente a execução da busca em-  
# A regra visualizada é: esquerda, nó atual, direita.  
  
passos_em_ordem = gerar_passos_visualizacao(raiz, "em")
```

```
mostrar_execucao_visual("Execução visual da busca em-ordem", passos,
```

Execução visual da busca em-ordem

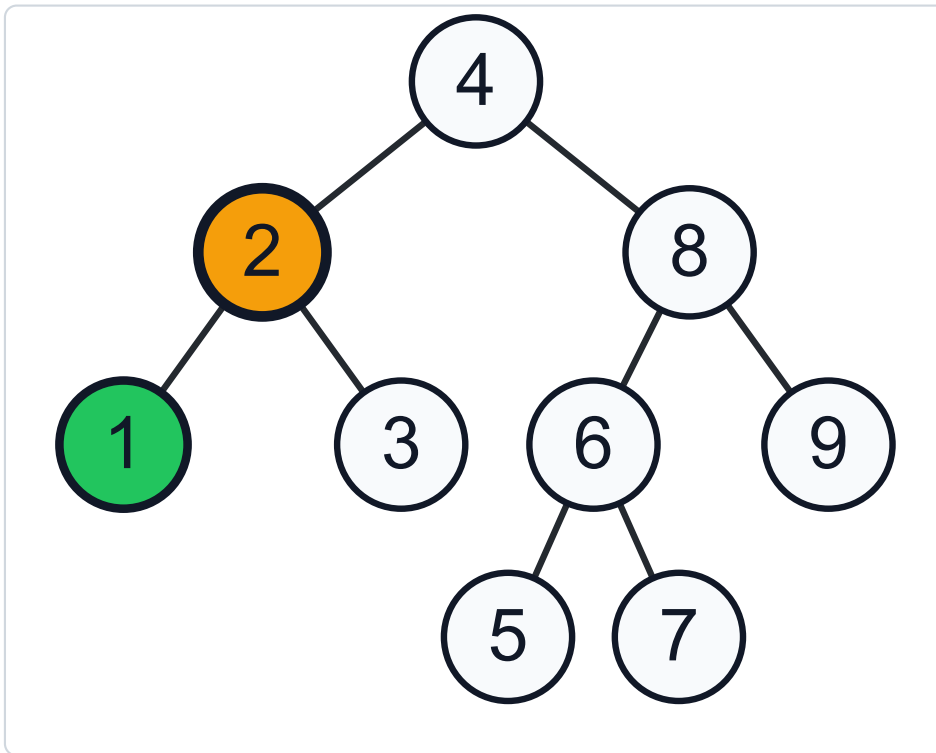
■ nó atual ■ já visitado □ ainda não visitado

Passo 1: visita 1



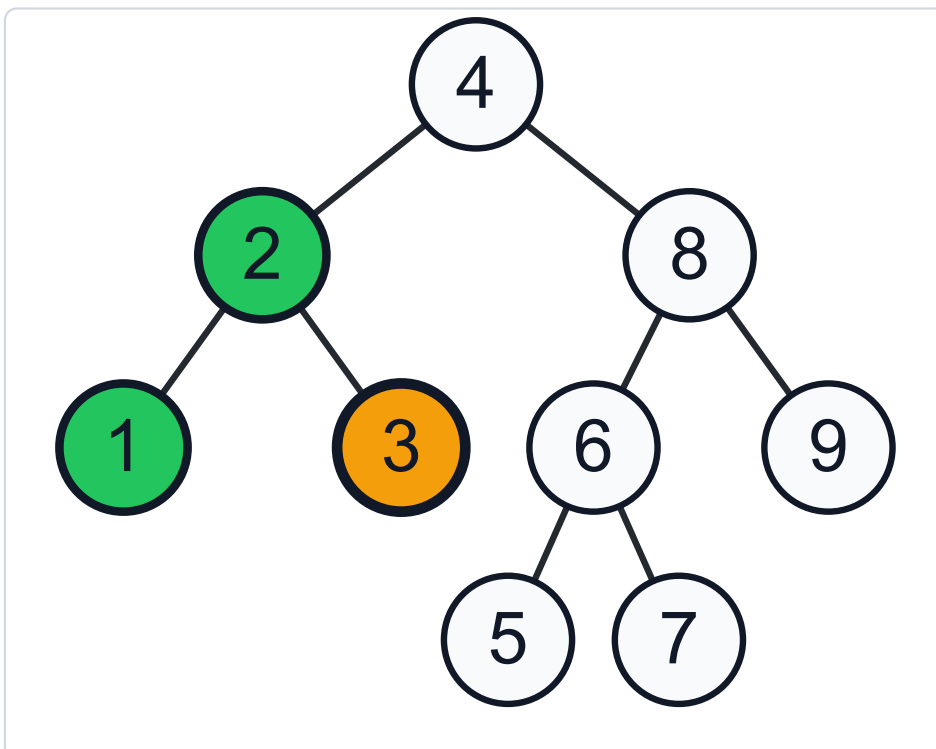
Sequência: 1

Passo 2: visita 2



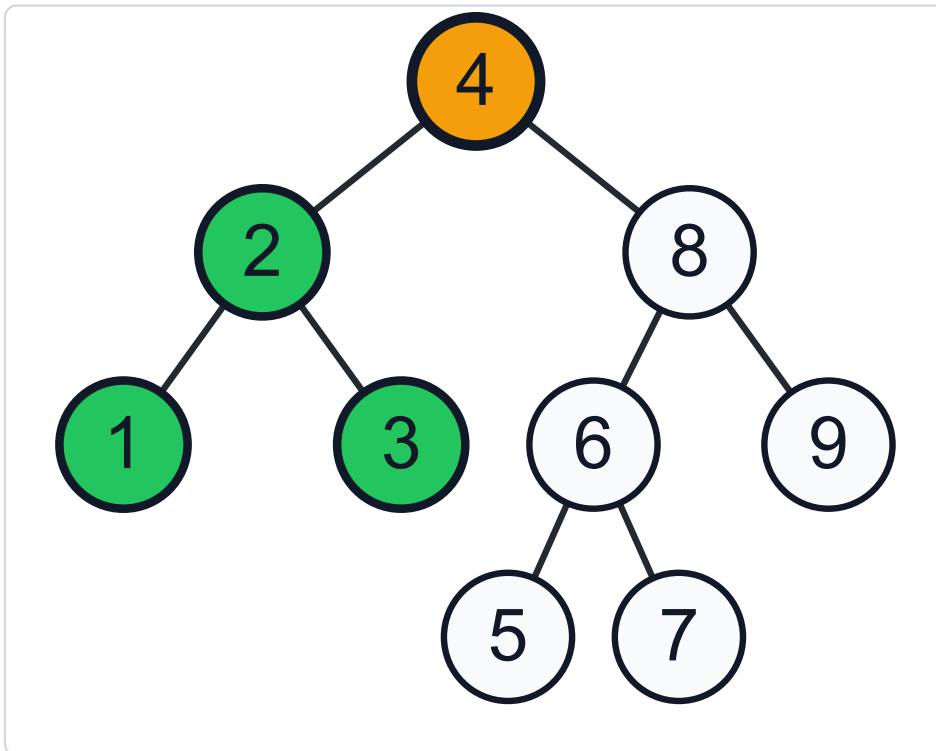
Sequência: 1, 2

Passo 3: visita 3



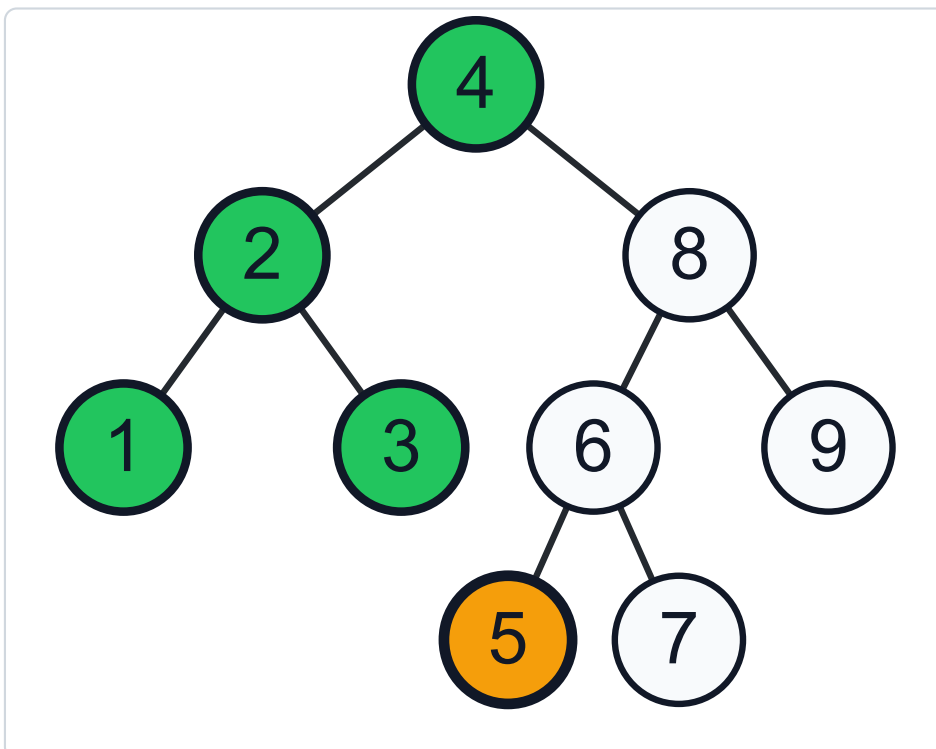
Sequência: 1, 2, 3

Passo 4: visita 4



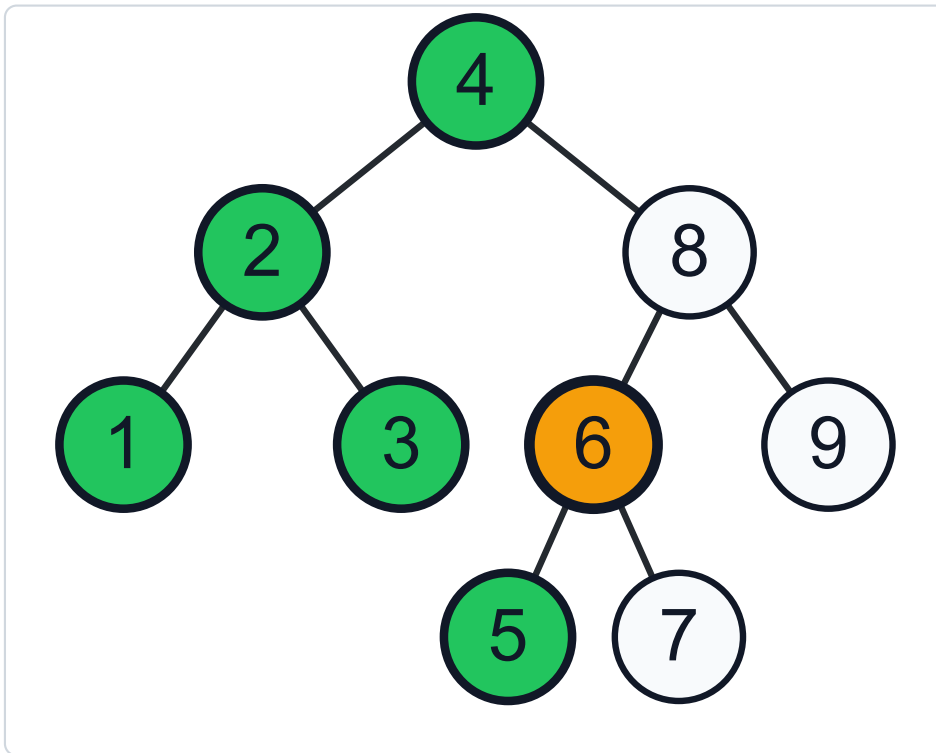
Sequência: 1, 2, 3, 4

Passo 5: visita 5



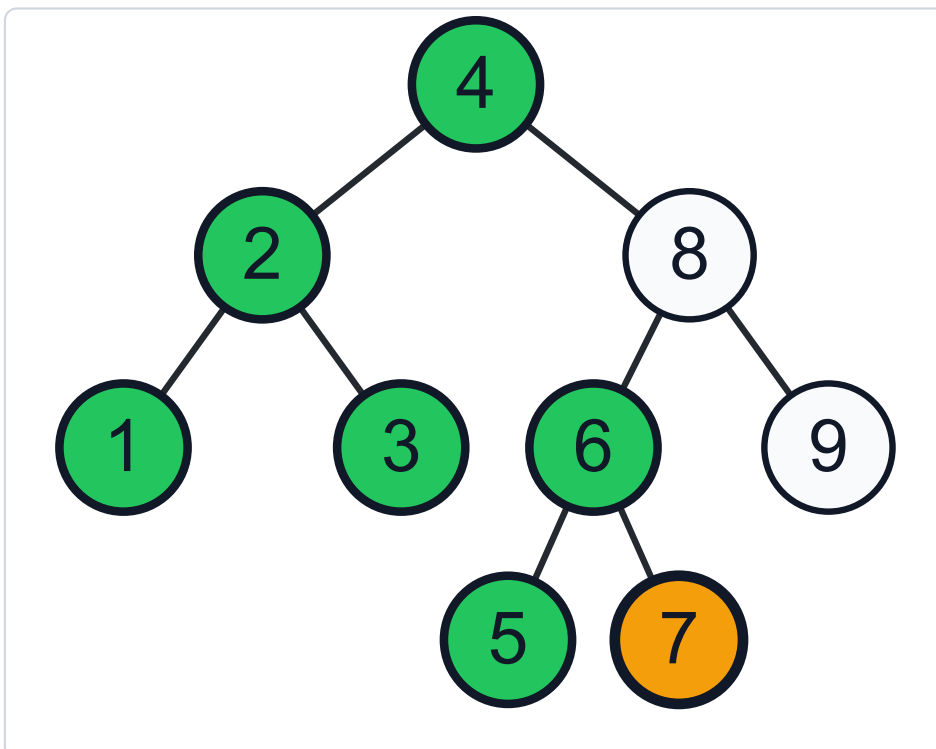
Sequência: 1, 2, 3, 4, 5

Passo 6: visita 6



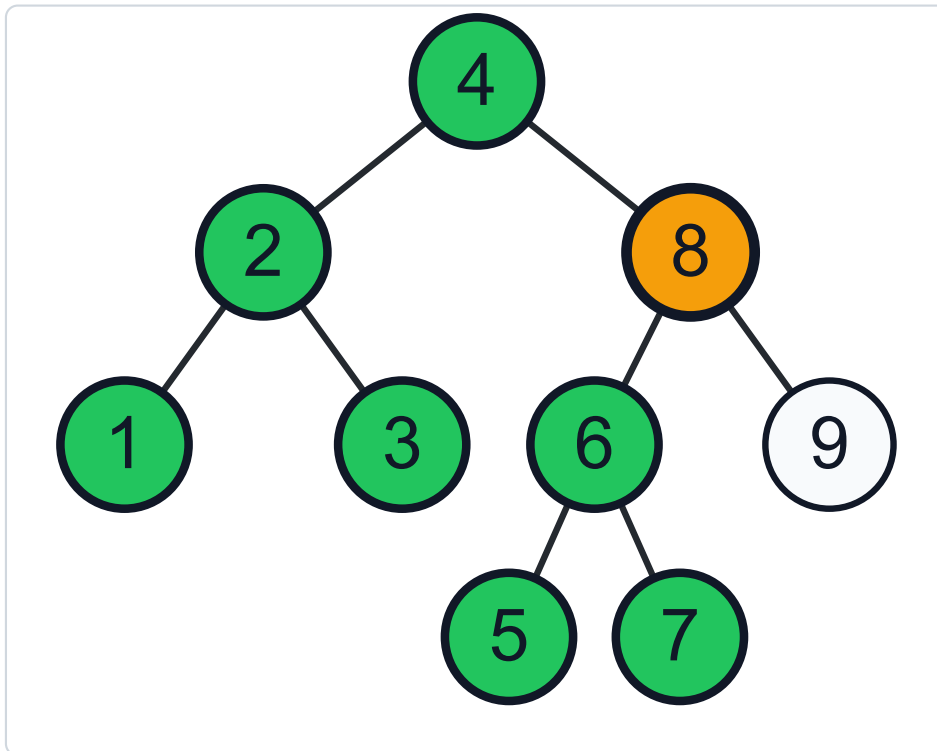
Sequência: 1, 2, 3, 4, 5, 6

Passo 7: visita 7



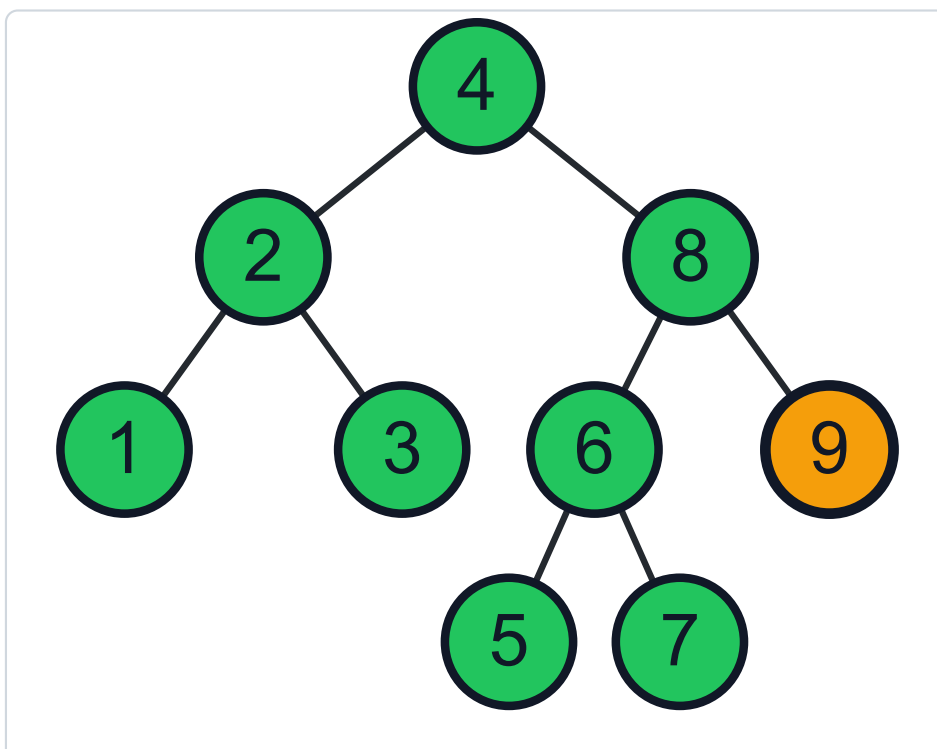
Sequência: 1, 2, 3, 4, 5, 6, 7

Passo 8: visita 8



Sequência: 1, 2, 3, 4, 5, 6, 7, 8

Passo 9: visita 9



Sequência: 1, 2, 3, 4, 5, 6, 7, 8, 9

```
In [12]: # Descrição: esta célula mostra visualmente a execução da busca pós
# A regra visualizada é: esquerda, direita, nó atual.

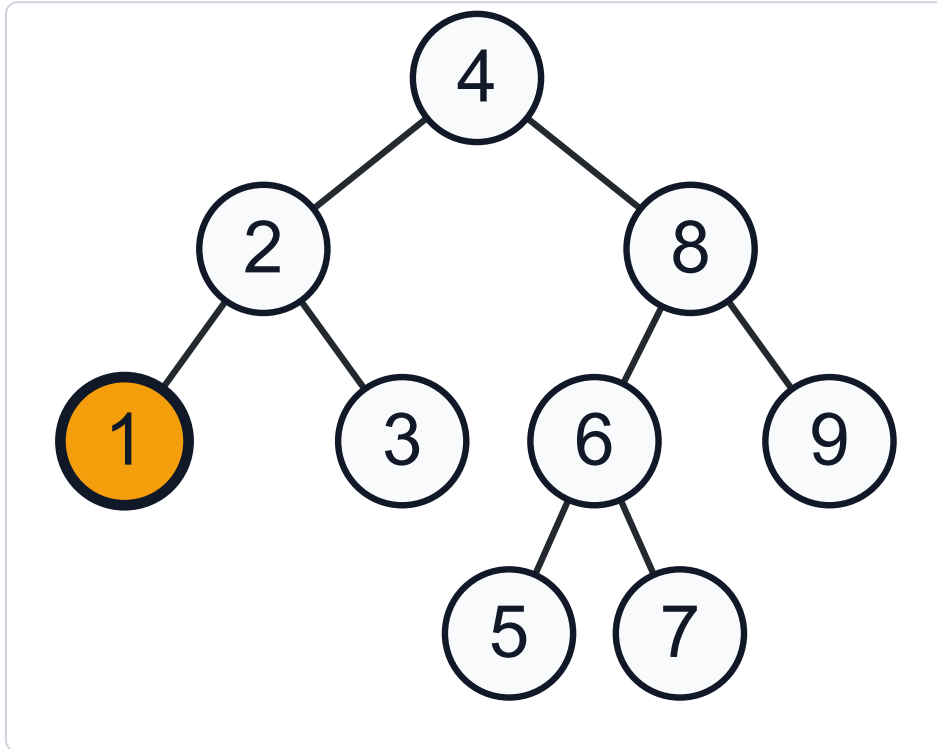
passos_pos_ordem = gerar_passos_visualizacao(raiz, "pos")
```

```
mostrar_execucao_visual("Execução visual da busca pós-ordem", passo
```

Execução visual da busca pós-ordem

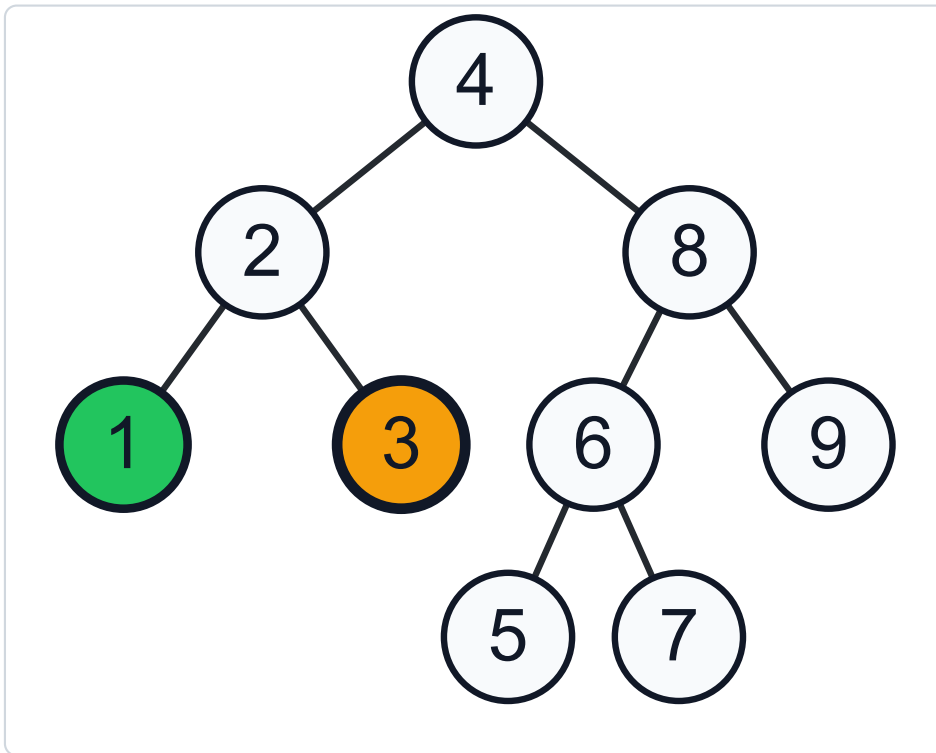
■ nó atual ■ já visitado □ ainda não visitado

Passo 1: visita 1



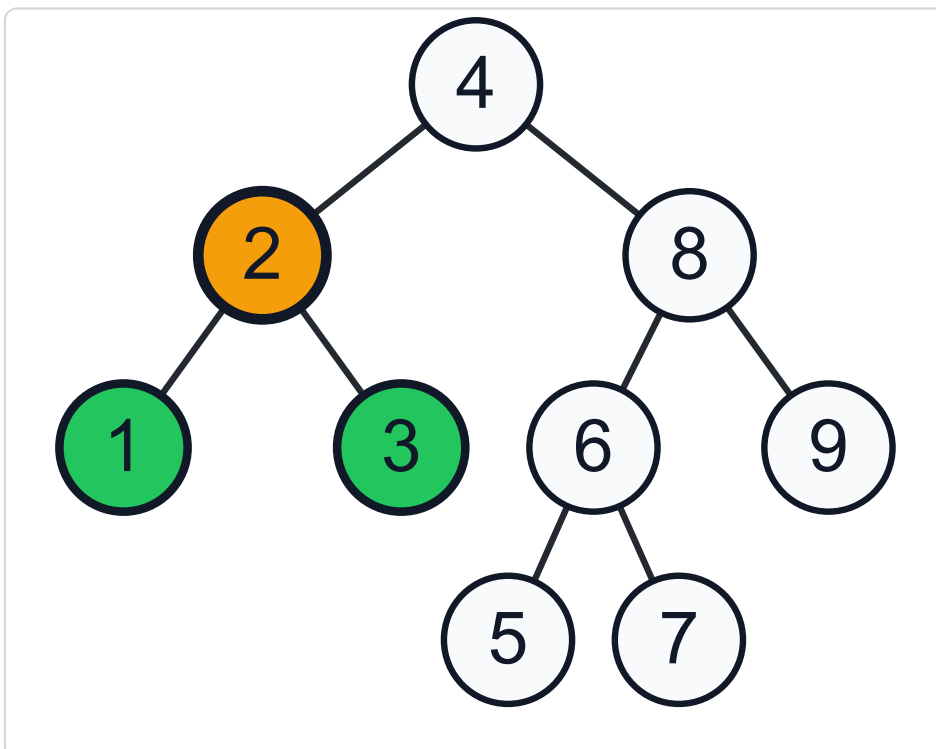
Sequência: 1

Passo 2: visita 3



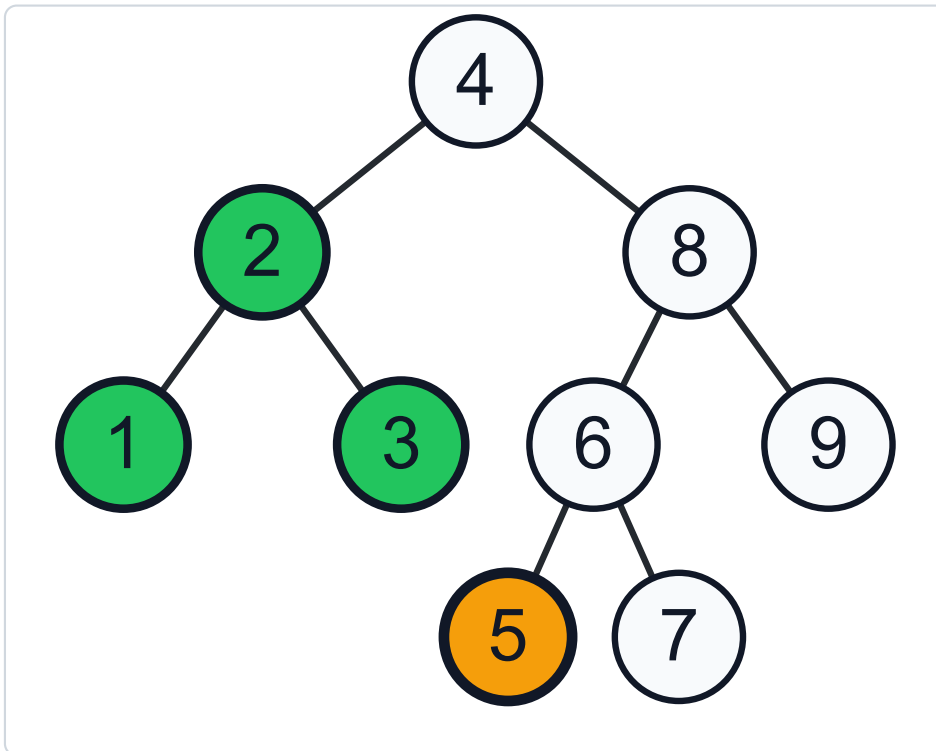
Sequência: 1, 3

Passo 3: visita 2



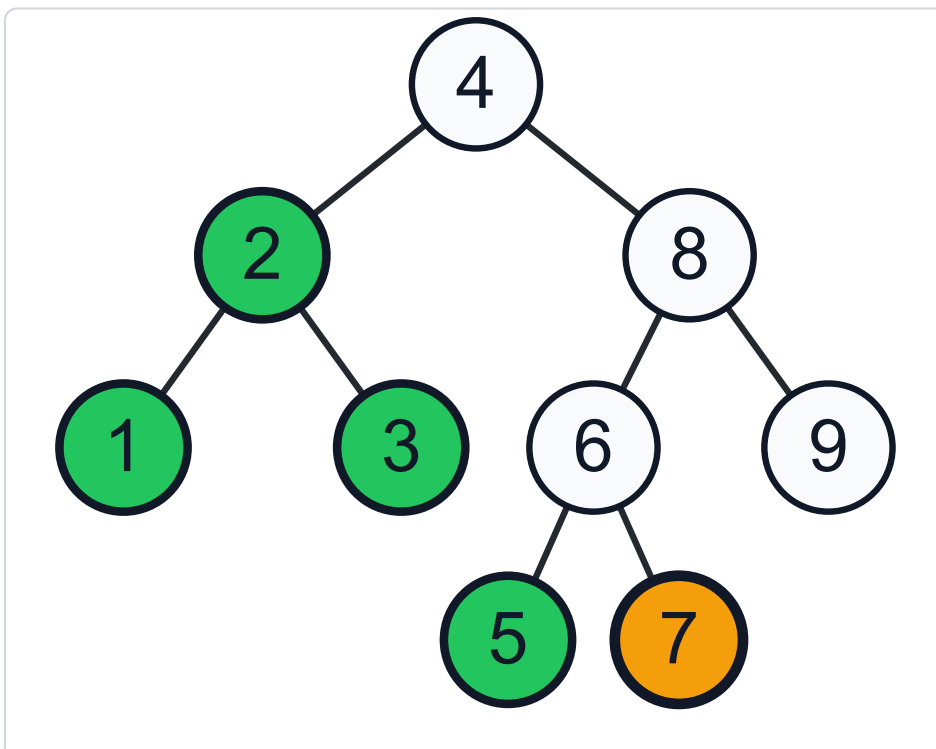
Sequência: 1, 3, 2

Passo 4: visita 5



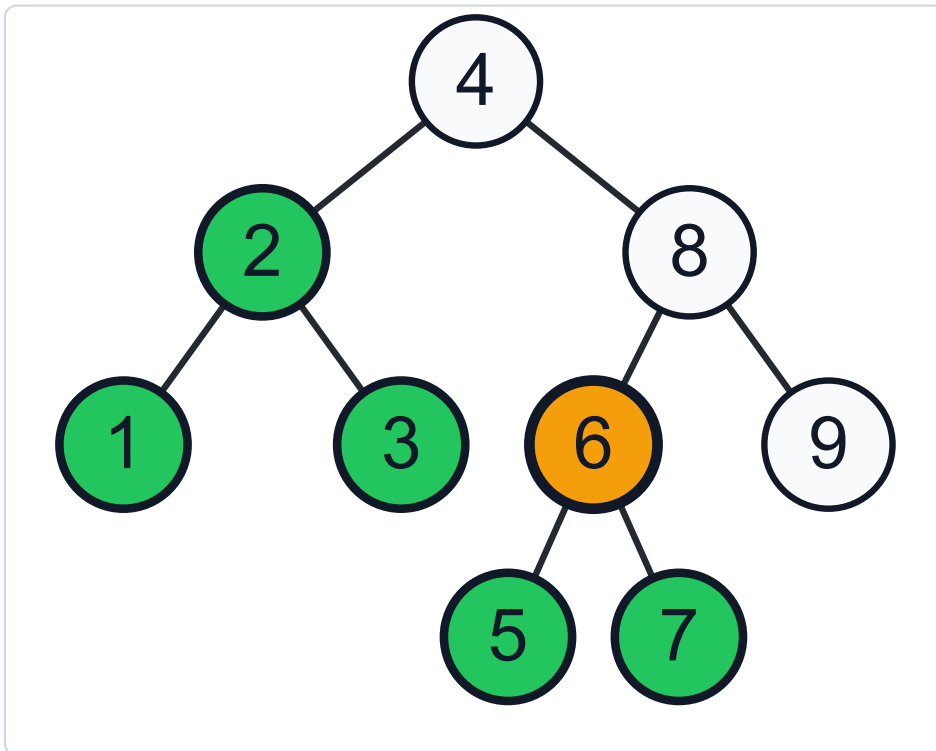
Sequência: 1, 3, 2, 5

Passo 5: visita 7



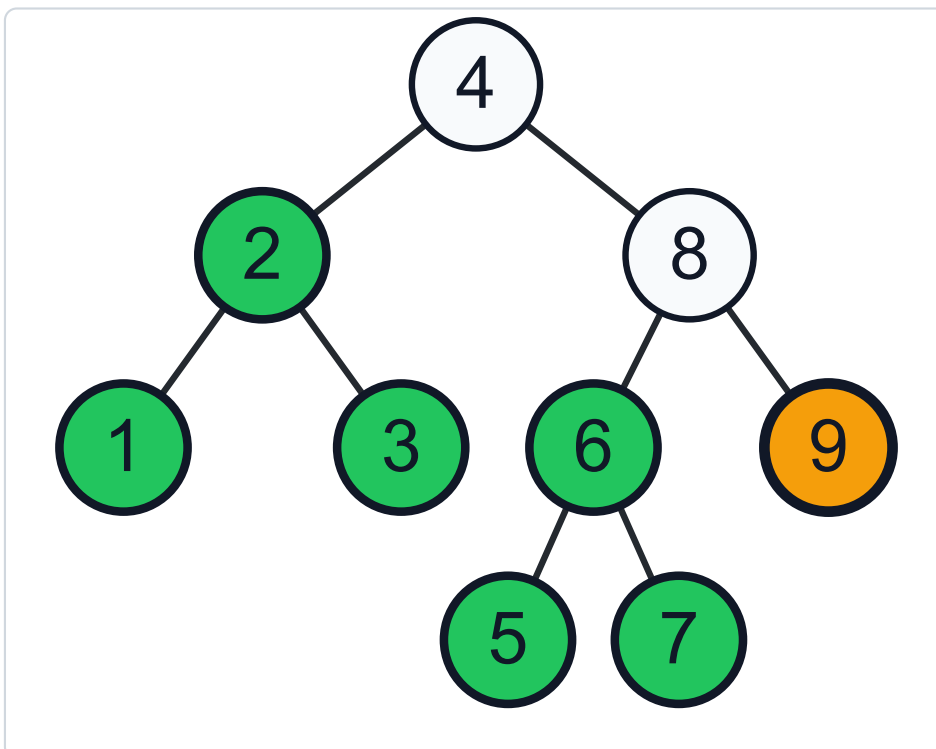
Sequência: 1, 3, 2, 5, 7

Passo 6: visita 6



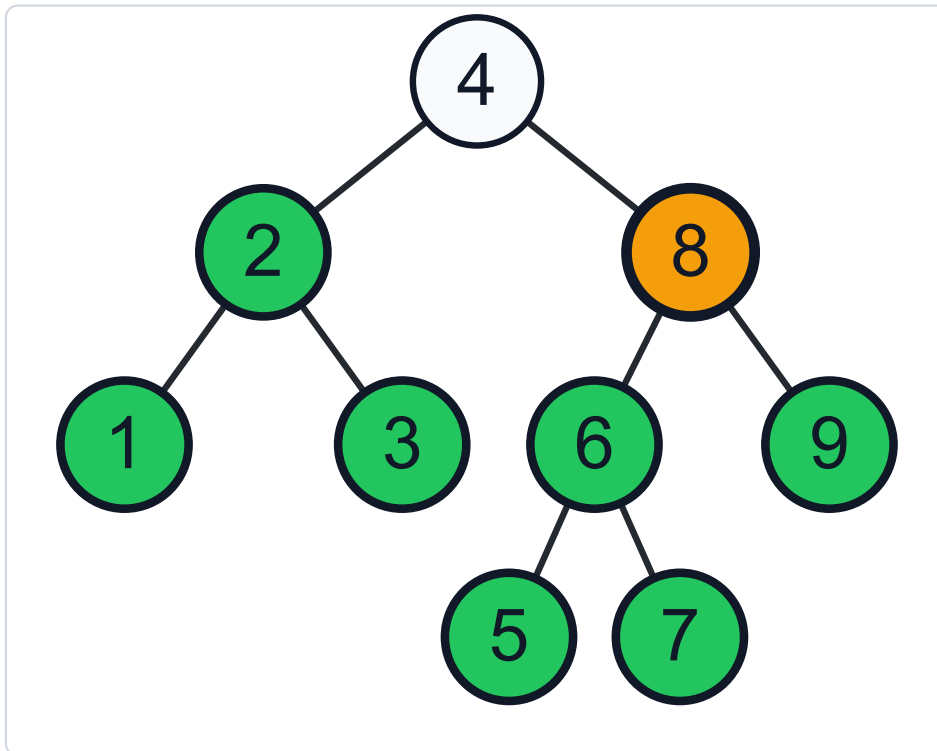
Sequência: 1, 3, 2, 5, 7, 6

Passo 7: visita 9



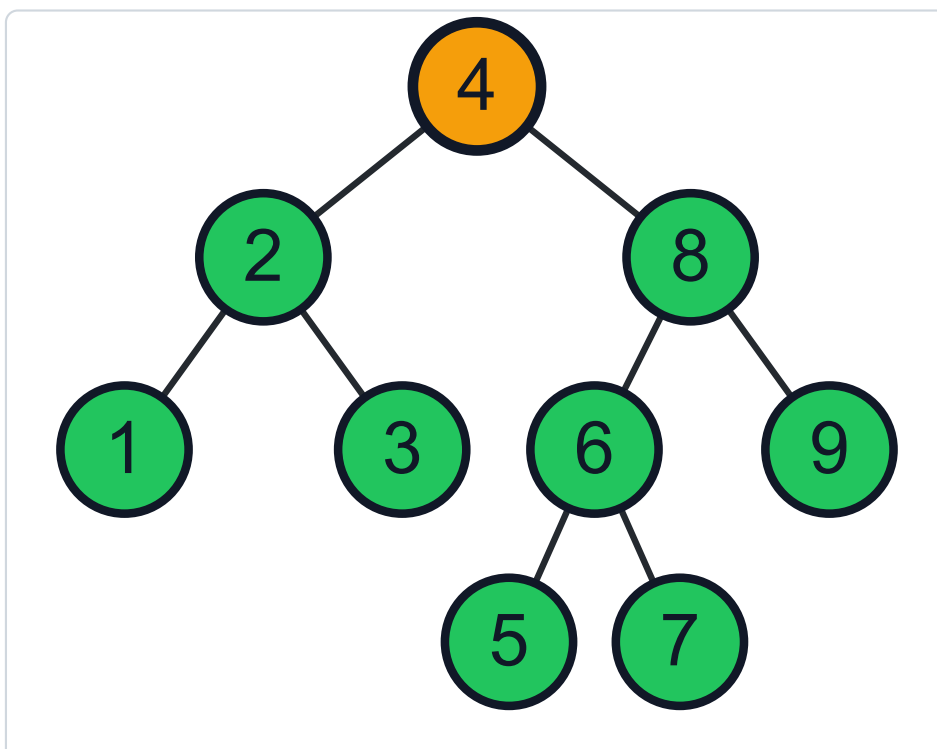
Sequência: 1, 3, 2, 5, 7, 6, 9

Passo 8: visita 8



Sequência: 1, 3, 2, 5, 7, 6, 9, 8

Passo 9: visita 4



Sequência: 1, 3, 2, 5, 7, 6, 9, 8, 4

Resultados pedidos no enunciado

Para a árvore binária apresentada no exercício, as saídas dos percursos em profundidade são:

- **Pré-ordem:** 4, 2, 1, 3, 8, 6, 5, 7, 9
- **Em-ordem:** 1, 2, 3, 4, 5, 6, 7, 8, 9
- **Pós-ordem:** 1, 3, 2, 5, 7, 6, 9, 8, 4